ML_Worksheet

Monday, April 27, 2026 6:54 PM

ML_Worksheet

ML TEAM — SPRING 2026

Practice Worksheet: Neural Networks · CNNs · Transformers · Attention · RL · Q-Learning

Name: Emma Ar Team:

Section 1 — Neural Networks (/ 20 pts)

Q1 [Calculation] A neuron has weights w = [3, -2, 1], bias b = -1, and input x = [2, 1, 4]. Compute the raw output z = w·x + b, then apply ReLU. Show your work.

z = 3×2 + (−2)×1 + 1×4 + (−1)
= 6 − 2 + 4 − 1 = 7
ReLU(z) = max(0, z) = 7

Q2 [Short Answer] A network has 3 layers with dimensions: input (784) → hidden (128) → hidden (64) → output (10). How many learnable parameters are in the weight matrices only (ignore biases)?

784×128 + 128×64 + 64×10 = 109,824 weights

Q3 [Explain] Without activation functions, a 10-layer neural network collapses to a single linear transformation. Explain why using matrix algebra.

Each layer adds one by. Stage matrix (W) and an bias (Wx+b) of layer. Without, the net Y=W_10(W_9(...W_1x)) multiplying gets one single linear transformation and one output. Pointless.

Q4 [Short Answer] You're training a classifier and notice the training loss is very low but validation loss is very high. Name this problem, and give two ways to fix it using what you know about neural networks.

Overfitting. ① Dropout. Randomly kill neurons during training, so the network can't memorize specific examples. ② Early stopping. Stop training when validation loss starts rising, before the model starts memorizing the training data and hurts its ability to generalize to new data.

Section 2 — Convolutional Neural Networks (/ 20 pts)

Q5 [Calculation] Apply the 1D filter W = [1, 0, -1] with stride 1 to input X = [3, 1, 4, 1, 5, 9]. Write the full output and explain what pattern this filter detects.

(3×1)+(1×0)+(4×−1) = −1
(1×1)+(4×0)+(1×−1) = 0
(4×1)+(1×0)+(5×−1) = −1
(1×1)+(5×0)+(9×−1) = −8

Output: [−1, 0, −1, −8]

Detects if the values ahead of a step higher than behind, where it's the most.

Q6 [Calculation] An image is 64×64. You apply: (1) Conv with 3×3 filter, stride 1, no padding. (2) 2×2 max pool, stride 2. (3) Conv with 3×3 filter, stride 1, no padding. What is the spatial size after each step?

① 62×62
② 31×31
③ 29×29

Q7 [Explain] Why do early CNN layers detect edges while deep layers detect semantic concepts like 'face' or 'wheel'? Explain in terms of receptive fields.

Early layers see tiny patches (3x3, maybe 5x5) pixels at a time, so they only detect simple things like edges, textures, or color changes. Deep layers stack many layers, so their receptive field grows large and can combine simple features into "face" or "wheel".

Q8 [Compare] A fully connected layer connecting 64×64 input to 1000 neurons has how many weights? A convolutional layer with 32 filters of size 3×3 on the same input has how many weights? Why is the difference important?

FC: 4,096 × 1,000 = 4,096,000 = 4.096 Mm weights.
Conv: 3×3×1×32 = 288 weights.

The difference is enormous (14,000 times). Conv works because the same filter works across different locations, sharing it weights. FC can't do that, treating each pixel differently.

Section 3 — Transformers & Attention (/ 20 pts)

Q9 [Explain] Consider: 'I saw the man with the telescope.' Explain how attention assigns different weights to 'man' and 'telescope' when computing the contextual embedding for 'with'. What does this tell us about the sentence's ambiguity?

The machine has to disambiguate, or ① I used the telescope to see the man. ② The man is the one with the telescope. Since the model compares a word against all the other words and learns what attention patterns matter, the weights it gives "with" depend on what it's looking for.

So it shows how different contexts for "saw/man/telescope". It tells us that the sentence isn't clearly about the telescope, ambiguity.

Q10 [Calculation] You have Q = [1, 0], K = [1, 0], V = [3, 1], [-1, 0]. d_k = 2. Compute the raw attention scores, scale by 1/√d_k, and apply softmax. Which key does Q attend to most?

Q·K^T = [1×1 + 0×0, 1×−1 + 0×0] = [1, −1]

s_0 = 1 / √2 ≈ 0.707
s_1 = −1 / √2 ≈ −0.707

softmax([0.707, −0.707]) ≈ [0.62, 0.38]

It attends to key one by 62% to 38%, most

Q11 [Short Answer] A transformer is trained on the sentence 'I love to eat french fries with ketchup.' During training it predicts every word simultaneously. Draw the causal mask as a 9×9 grid (use 1 = attend, 0 = masked) for the first 4 tokens.

	I	love	to	eat	→ the model
I	1	0	0	0	
love	1	1	0	0	
to	1	1	1	0	
eat	1	1	1	1	

Q12 [Compare] BERT uses bidirectional attention. GPT uses causal (left-to-right) attention. For each model, explain which task it is better suited for and why the masking choice directly causes this.

BERT: bidirectional attention, it sees tokens forward. So every token knows its full context. Better for understanding tasks (classification, sentiment, or answering questions) where knowing what's ahead helps.

GPT: left-to-right attention, it only looks backward. So every token can't see what's ahead. Forced to generate one token at a time, left to right. Better for open-ended generation, where you don't know the ending in advance.

Section 4 — Reinforcement Learning (/ 20 pts)

Q13 [Calculation] γ = 0.8. An agent receives rewards r_t=0, r_t+0, r_t+5, r_t+0, r_t+10 then the episode ends. Compute the discounted return G_t starting from t=0.

G_t = r_t + γ·r_{t+1} + γ²·r_{t+2} + γ³·r_{t+3} + γ⁴·r_{t+4}

Q14 [Short Answer] Define the Markov property. Give one example of a real-world problem where the Markov property holds, and one where it does NOT hold.

Markov property: The future depends only depends on the current state.

Holds: Chess or Go, where the board tells you everything you need to know about the next move.

Doesn't hold: Driving in traffic at night, because the Markov assumption cannot be met.

Q15 [Explain] What is the credit assignment problem? Use a chess analogy. How does the Bellman equation help solve it?

Credit assignment problem is when you win after 4 moves, but you don't know which move led the most impact.

In chess/go/shogi, the Bellman equation, iteratively back-propagates value in reverse through the moves.

Each move inherits value from the next one. It starts by assigning terminal rewards a discounted value of the next.

Over, this allows to be confident each move.

Q16 [Compare] An agent uses ε-greedy with ε = 0.1. (a) What fraction of actions are random? (b) If the agent is near a cliff with −100 reward, what is the expected penalty per step from random actions? (c) Should ε be higher early or late in training? Why?

(a) 10% = 0.1

(b) 0.1 × (−100) = −10 per step. Random actions can't distinguish safe from cliff. Random walk near cliff is dangerous.

(c) Early, because the agent doesn't know anything yet. Exploration is key. Decay over time, so it stops exploring once it knows enough.

Section 5 — Q-Learning (/ 20 pts)

Q17 [Calculation] Current Q(s,a) = 4. You observe r = 8, next state s' has Q(s', left) = 6 and Q(s', right) = 10. α = 0.3, γ = 0.9. Compute the new Q(s,a). Show the TD target and TD error.

TD target = r + γ·max(Q(s', left), Q(s', right))
= 8 + 0.9 × 10 = 17

TD error = TD target − Q(s,a)
= 17 − 4 = 13

Q18 [Trace] 4-state chain: S0 --S1 --S2 --S3(terminal). r(S2 --> S3)=20, all others 0. α=0.5, γ=1.0, Q initialized to 0. Trace Episodes 1 and 2 fully. Fill in the table below.

Step	Transition	Reward	Q Update Formula	Q(S1)	Q(S2)
Ep1 Step1 (S0 --S1)	S0 --> S1	0	α × (r + γ·Q(s',a') − Q) = 0.5 × (0 + 1·0 − 0) = 0	0	0
Ep1 Step2 (S1 --S2)	S1 --> S2	0	α × (r + γ·Q(s',a') − Q) = 0.5 × (0 + 1·0 − 0) = 0	0	0
Ep1 Step3 (S2 --S3)	S2 --> S3	20	α × (r + γ·Q(s',a') − Q) = 0.5 × (20 + 1·0 − 0) = 10	0	10
Ep2 Step1 (S0 --S1)	S0 --> S1	0	α × (r + γ·Q(s',a') − Q) = 0.5 × (0 + 1·0 − 0) = 0	0	0
Ep2 Step2 (S1 --S2)	S1 --> S2	0	α × (r + γ·Q(s',a') − Q) = 0.5 × (0 + 1·10 − 0) = 5	5	0
Ep2 Step3 (S2 --S3)	S2 --> S3	20	α × (r + γ·Q(s',a') − Q) = 0.5 × (20 + 1·0 − 5) = 15	5	15

Q19 [Short Answer] Q-learning is called 'off-policy.' What does this mean? Write out the SARSA update rule and explain the one key difference from Q-learning's update rule.

Off-policy means that the policy used to generate the data (how the agent actually acts) is not always the same as the policy being learned.

Q-learning update: Q(s,a) ← Q(s,a) + α[r + γ·max(Q(s',a')) − Q(s,a)]
SARSA update: Q(s,a) ← Q(s,a) + α[r + γ·Q(s',a') − Q(s,a)]

SARSA doesn't assume that you're using the greedy policy. Q-learning is off-policy.

Q20 [Explain] A Q-table has 10,000 states and 5 actions. Describe one situation where this is fine and one where this is completely unworkable. What is the standard fix for the unworkable case?

Fine: Small grid world (about 100 squares) where the agent visits most of the states.

Unworkable: Atari games or Go (the board position alone has 10,000 possible states).

Standard fix: Replace the table with a neural network to map a state-action pair to a Q-value, rather than storing every single value in a table.